of the Jedi Masters

Edward Hartnett, CIRES/NOAA 12/3/24

TABLE OF CONTENTS

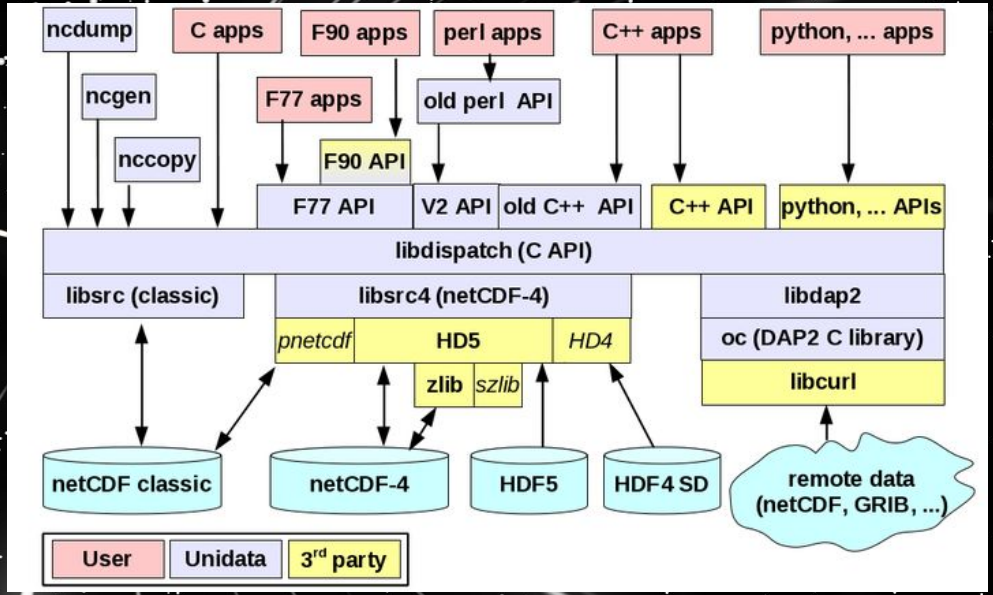
INSTALLATION

- **Both CMake and autotools build are available.**
- **They each support the same features (mostly).**
- **The autotools build will be dropped in a future netCDF release.**
- **Some helpful options:**
  - **–enable-parallel-tests (requires MPI build)**
  - **–with-mpiexec= (allows us to use srun instead of mpiexec)**
  - **–with-plugin-dir= (where plugins should be installed)**
  - **–enable-logging (turns on logging capability in netcdf-c)**
  - **–enable-hdf4 (read HDF4 SD files as if they were netCDF)**
  - **–disable-remote-functionality (turns off DAP and Zarr)**

Always run the netcdf-c and the
netcdf-fortran tests for every install.

FORMATS

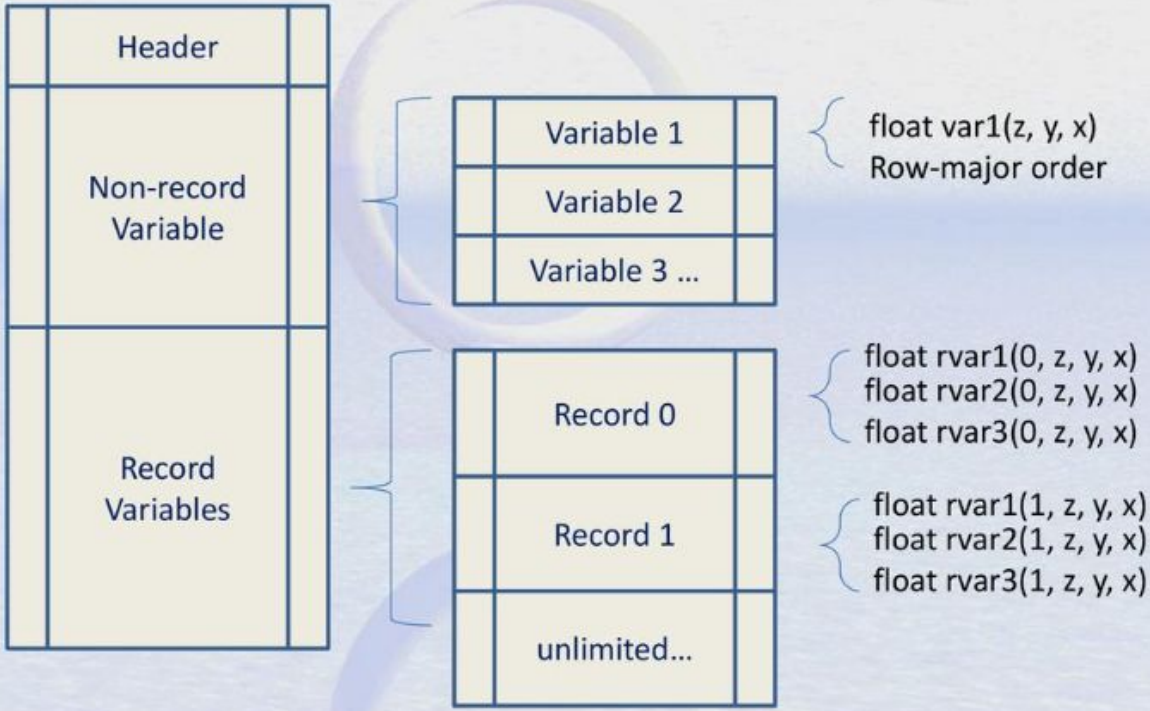
- There are several different binary formats. All are equally netCDF files.
- Some formats are optionally supported.
- The User-defined Format feature allows new plugins to be written, make any data format readable to netCDF.
- Format only has to be identified at write time, it's figured out automatically for reads.
- Learn the format with nc_inq_format(), or even more with nc_inq_format_extended().



Binary Formats

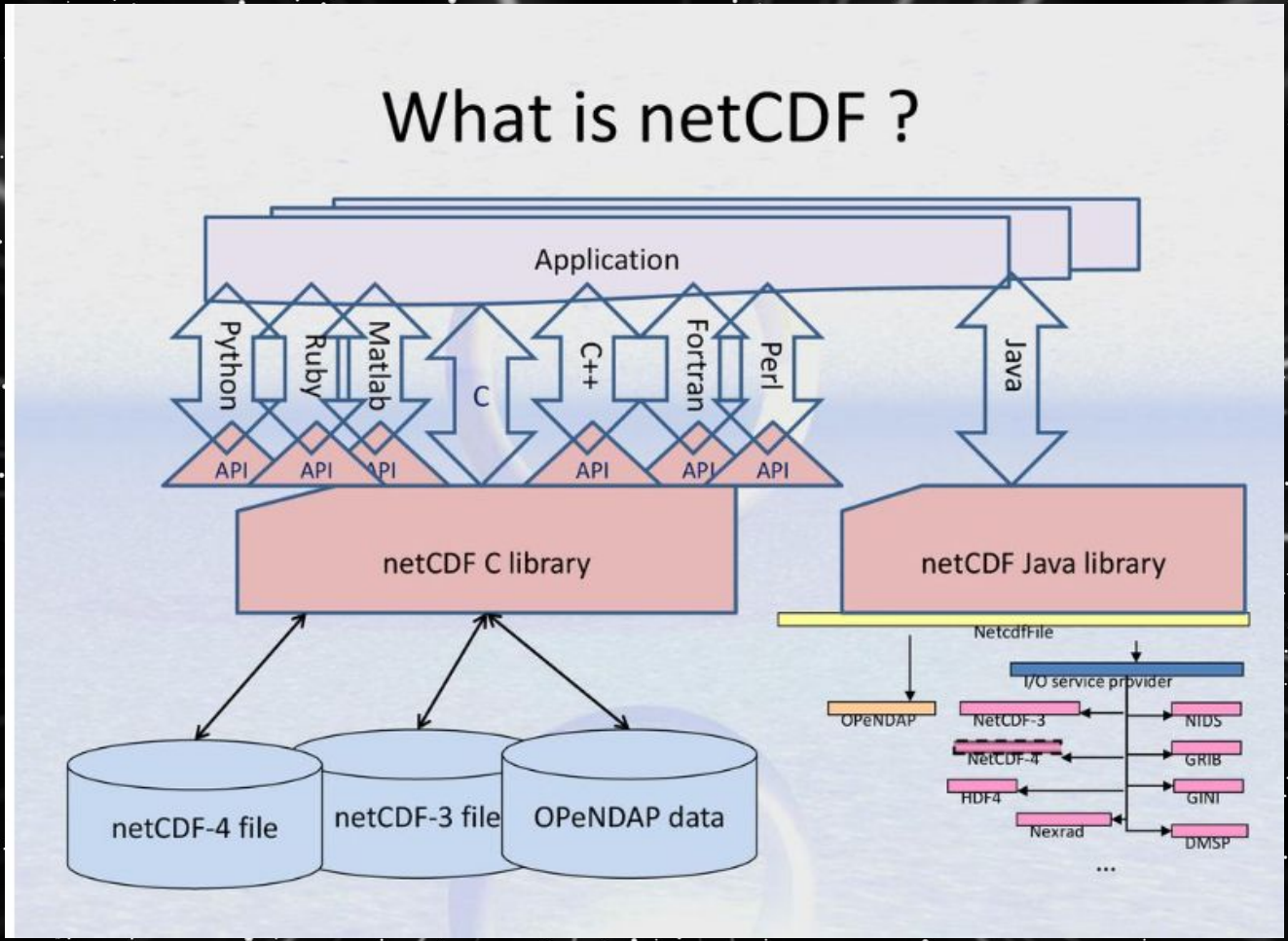
Format	nc_create()	nc_inq_format()	Notes
classic	none	NC_FORMAT_CLASSIC	Original netCDF format. Many 2/4 GB limits. Still and always the default format.
64-bit-offset	NC_64BIT_OFFSET	NC_FORMAT_64BIT_OFFSET	Like classic, but less 32-bit limits. Obsolete, use 64-bit-data format (CDF5) instead.
HDF5 classic data model	NC_NETCDF4| NC_CLASSIC_MODEL	NC_FORMAT_NETCDF4_CLASSIC	Data model of classic format, but no 32-bit limits, plus compression.
HDF5 extended data model	NC_NETCDF4	NC_FORMAT_NETCDF4	Full HDF5 data model. Non-netCDF HDF5 files can also be read.
64-bit-data (a.k.a. CDF5)	NC_CDF5	NC_FORMAT_64BIT_DATA	Like classic, but no 32-bit limits, contributed by pnetcdf (parallel-netcdf) project.
HDF4 SD (read only)	N/A	Unknown	Must include HDF4 at build time. Read-only.
Zarr	Use URL for filename	Unknown	Can read or write netCDF-4 extended model, without user-defined types, as Zarr data on Amazon S3 storage.

NetCDF-3 file format



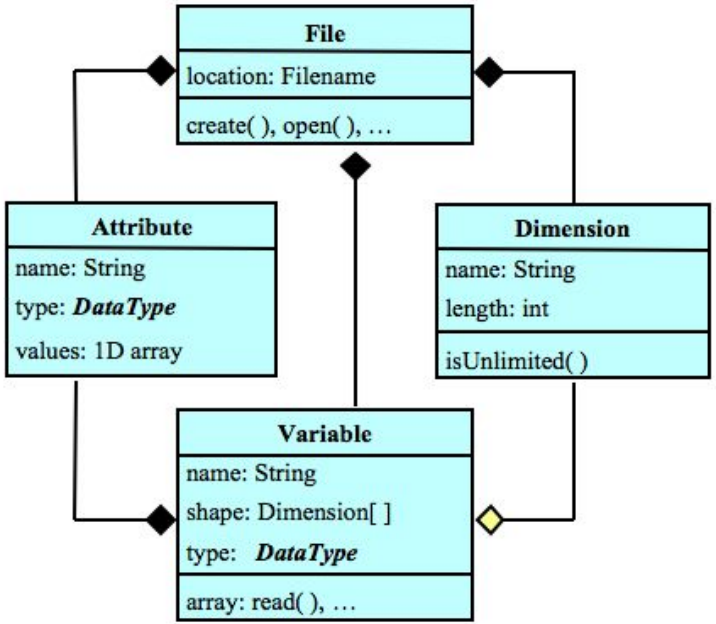
From What is NetCDF, Caron, 2009

What is netCDF ?



From What is NetCDF, Caron, 2009

Original (Classic) Data Model

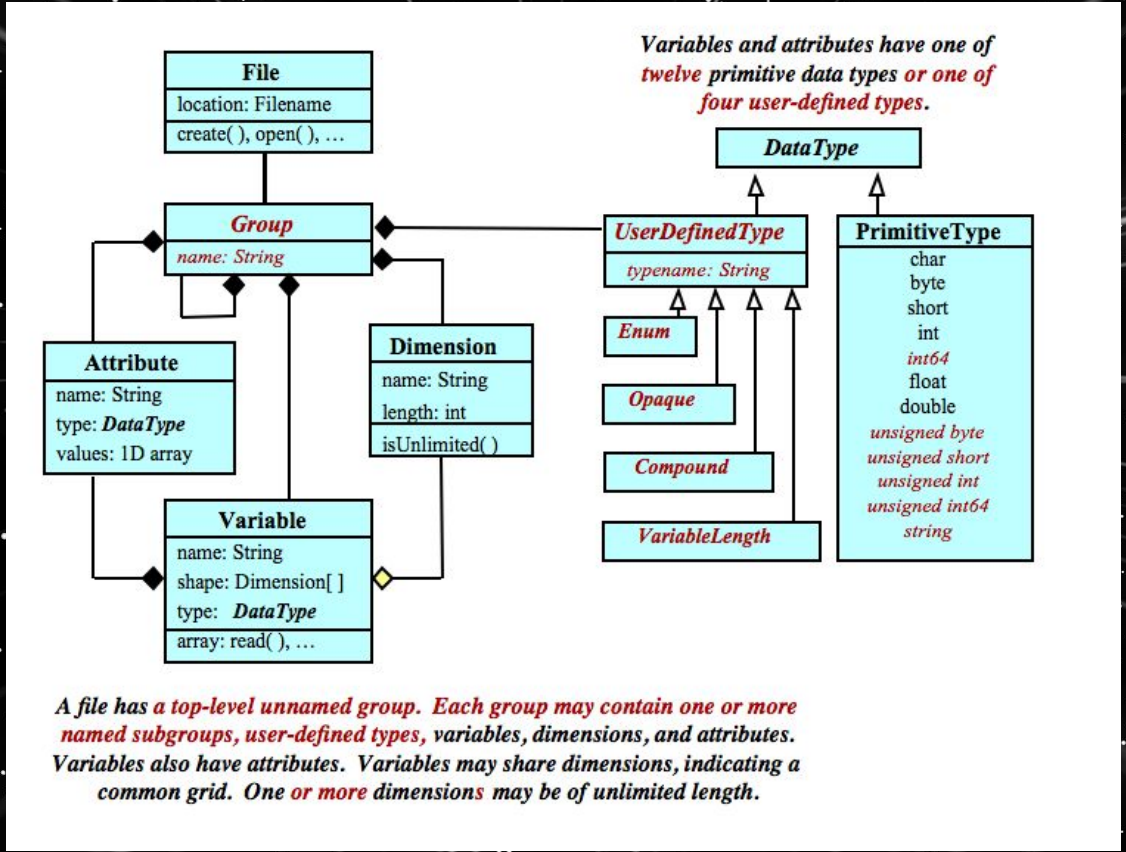


Variables and attributes have one of six primitive data types.

DataType
char
byte
short
int
float
double

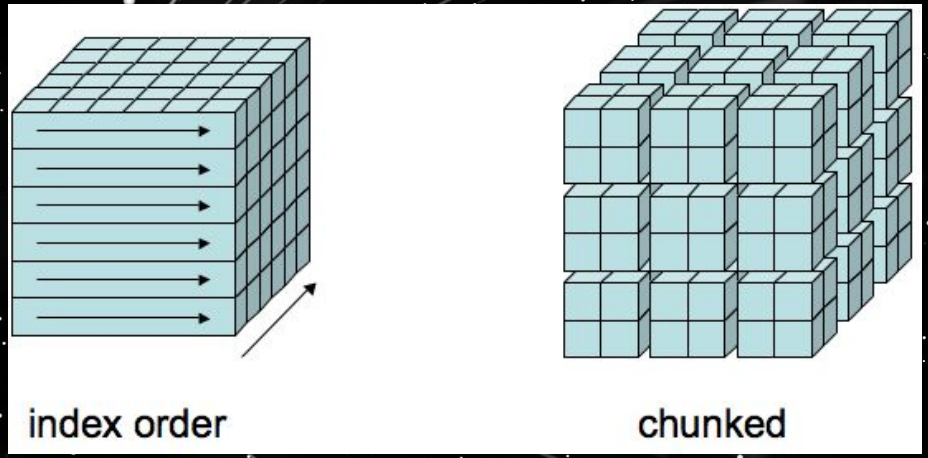
A file has named variables, dimensions, and attributes. Variables also have attributes. Variables may share dimensions, indicating a common grid. One dimension may be of unlimited length.

NetCDF-4 (Extended) Data Model



Chunking

- Most HDF5 data are chunked.
- Chunking is required for any form of compression, and for variables with unlimited dimension.
- Try and end up with squarish chunks.
- Big chunks are better.
- Beware of default chunking for vars with unlimited dimension.
- Match chunksizes to read or write patterns (whichever needs to be faster).
- Properly chunked HDF5 data can be read faster than classic format data. If not, then change your chunksizes.



Filters

- NetCDF supports all HDF5 filters.
- Filters are plugins that can provide new capabilities (mostly compression).
- Filters must be installed in plugin directory.
- NetCDF/HDF5 can learn plugin directory from environment variable or it can be set programmatically.
- Using a compression filter other than zstd and zlib is possible with some effort.
- Doing this from C is easier than from Fortran.
- I hope to add native support for the LZ4 filter in 2025, to provide even faster compression.

Commitment to Backward Compatibility

Because preserving access to archived data for future generations is sacrosanct :

- **Data access**: New netCDF software will provide read and write access to ***all*** earlier forms of netCDF data.
- **APIs and programs**: Existing C, Fortran, and Java netCDF programs will be supported by new netCDF software (possibly after recompiling).
- **Commitment**: Future versions of netCDF software will continue to support data access, API, and conventions compatibility.

User-Defined Format

- Plugins can be written for any format.
- Plugin is a library which implements a subset of netCDF core functions.
- Plugin translates between another format and netCDF's internal data model.
- Plugin can provide read-only or read-write access.
- With plugin, any format can be read or written as if it were netCDF.
- The plugin code code HDF4 in the netcdf-c distribution provides a compact example.

DAP

- Remote access to netCDF files on an OPeNDAP server.
- Server must be running OPeNDAP software.
- Remote programs can then subset the data with netCDF nc_get_vara() calls. Only the selected data will be transmitted over the internet.
- To the calling program, looks exactly like a disk netCDF file.
- Used extensively by NASA.
- Can read many formats and present them as netCDF.

https://www.opendap.org/

ZARR

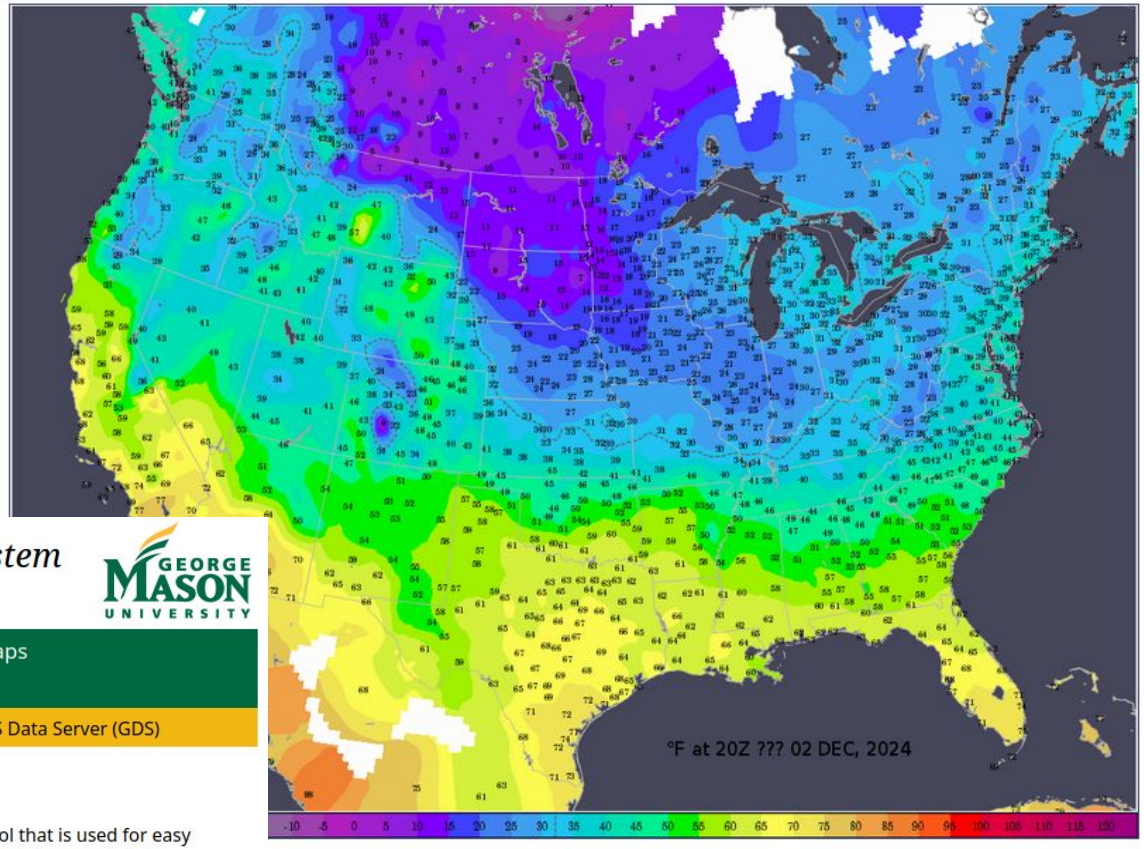
- “Zarr is a community project to develop specifications and software for storage of large N-dimensional typed arrays, also commonly known as tensors. A particular focus of Zarr is to provide support for storage using distributed systems like cloud object stores, and to enable efficient I/O for parallel computing applications.”
- NetCDF programs can read/write to Zarr without change of code.
- Using this, existing netCDF applications (including Fortran) can be easily converted to the cloud.
- Only the filename changes. All other netCDF calls remain the same.

NCO TOOLS

“The NCO toolkit manipulates and analyzes data stored in netCDF-accessible formats, including DAP, HDF4, HDF5, and, most recently, Zarr. NCO exploits the geophysical expressivity and logic of many CF (Climate & Forecast) metadata conventions, the flexible description of physical dimensions translated by UDUnits, the network transparency of OPeNDAP, the storage capabilities (e.g., compression, chunking, groups) of HDF (the Hierarchical Data Format), many powerful mathematical and statistical algorithms of GSL (the GNU Scientific Library), and the lossy and lossless compression codecs provided by the CCR (Community Codec Repository). NCO is fast, powerful, and free.”

- `ncap2` netCDF Arithmetic Processor (examples)
- `ncatted` netCDF ATTribute EDitor (examples)
- `ncbo` netCDF Binary Operator (addition, multiplication...) (examples)
- `ncchecker` netCDF Compliance CHECKER (examples)
- `ncclimo` netCDF CLIMatOlogy Generator (examples)
- `nces` netCDF Ensemble Statistics (examples)
- `ncecat` netCDF Ensemble conCATenator (examples)
- `ncflint` netCDF FiLe INTerpolator (examples)
- `ncks` netCDF Kitchen Sink (examples)
- `ncpdq` netCDF Permute Dimensions Quickly, Pack Data Quietly (examples)
- `ncra` netCDF Record Averager (examples)
- `ncrcat` netCDF Record conCATenator (examples)
- `ncremap` netCDF REMAPer (examples)
- `ncrename` netCDF RENAMEer (examples)
- `ncwa` netCDF Weighted Averager (examples)

Latest Surface Temperature Analysis



COLA

Grid Analysis and Display System (GrADS)

GEORGE MASON UNIVERSITY

COLA | Climate Dynamics PhD | GrADS | Weather Maps | Virginia Weather

Downloads | Documentation | Users Forum | GrADS Data Server (GDS)

Overview of GrADS

The Grid Analysis and Display System (GrADS) is an interactive desktop tool that is used for easy access, manipulation, and visualization of earth science data. GrADS has two data models for handling gridded and station data. GrADS supports many data file formats, including binary (stream or sequential), GRIB (version 1 and 2), NetCDF, HDF (version 4 and 5), and BUFR (for station data). GrADS has been implemented worldwide on a variety of commonly used operating systems and is freely distributed over the Internet.

Parallel I/O

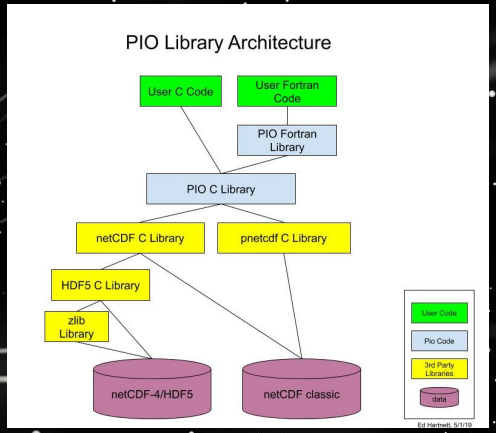
- For classic format files, parallel-netcdf (a.k.a. pnetcdf) provides parallel I/O.
- NetCDF can use parallel-netcdf for parallel I/O (must be enabled at build-time).
- NetCDF can use HDF5 for parallel I/O on NetCDF-4 or HDF5 files.
- Using the native netCDF API, each process must keep track of it's offsets in the global data space.

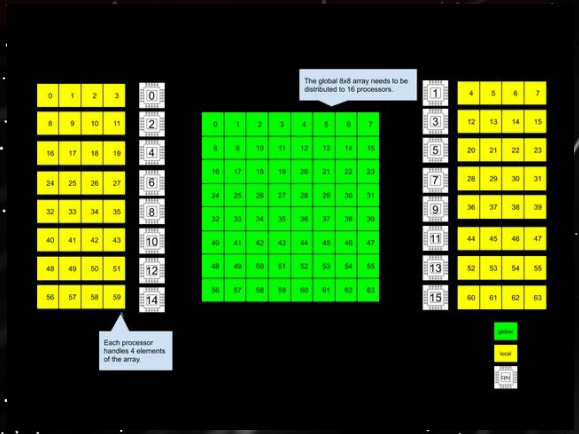
```
/* Set up slab for this process. */
start[0] = 0;
start[1] = mpi_rank * DIMSIZE/mpi_size;
count[0] = DIMSIZE2;
count[1] = DIMSIZE/mpi_size;

/* Create some test data. */
data      = malloc(sizeof(int)*count[1]*count[0]);
tempdata  = data;
for (j = 0; j < count[0];j++){
  for (i=0; i < count[1]; i++){
    *tempdata = mpi_rank*(j+1);
    tempdata ++;
  }
}

/* Write two dimensional integer data */
if ((res = nc_put_vara_int(ncid, nvid, start, count, data))){
   free(data);
   BAIL(res);
}
```

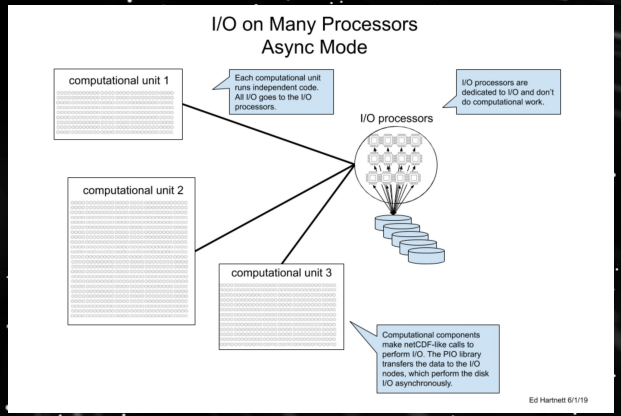
Use PIO for More Complex Tasks

- **Use PIO for more advanced access. With PIO, process does not need to keep track of offsets.**
- **PIO can integrate with netCDF so that existing netCDF metadata code works with PIO.**

Before writing complex I/O handling on HPC system, look at PIO features.







Future

- netcdf-c-3.9.3 will be released soon, with bugfixes and better support for filter path setting.
- Faster compression with LZ4 is coming in 2025.
- More work on Zarr.
- Support for complex types?
- Support for 16-bit float?

School of NetCDF